

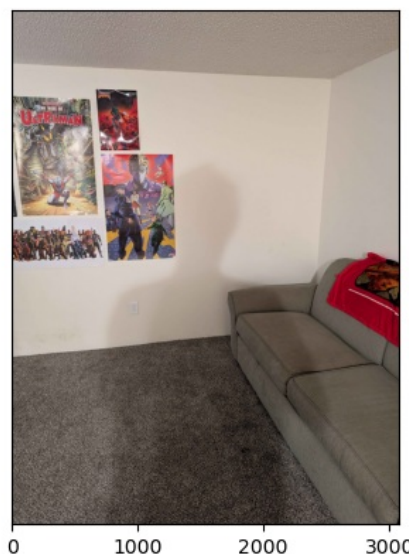
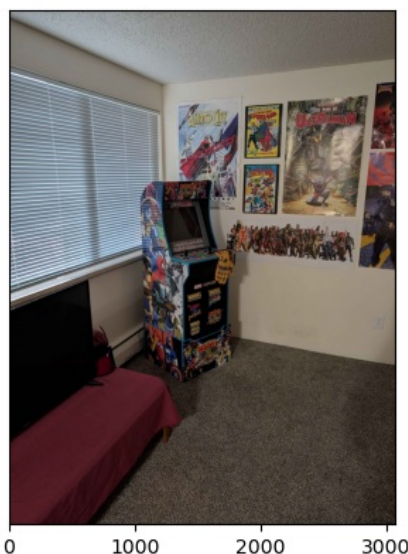
```
In [2]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from skimage import transform
```

Exercise 1

```
In [3]: img1 = cv.cvtColor(cv.imread('./hw10_image/ex1_1.jpg'), cv.COLOR_BGR2RGB)
img2 = cv.cvtColor(cv.imread('./hw10_image/ex1_2.jpg'), cv.COLOR_BGR2RGB)
img3 = cv.cvtColor(cv.imread('./hw10_image/ex1_3.jpg'), cv.COLOR_BGR2RGB)

imgs = [img1, img2, img3]

fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(12,5))
for i in range(3):
    ax[i].imshow(imgs[i])
    ax[i].get_yaxis().set_visible(False)
plt.show()
```



```
In [4]: def find_match(des1, des2, ratio=0.75):
    good_matches = []
    # match features using FLANN matcher
    FLANN_INDEX_KDTREE = 1
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
    search_params = dict(checks=50)
    matcher = cv.FlannBasedMatcher(index_params, search_params)
    matches = matcher.knnMatch(des1, des2, k=2)

    # Lowe's ratio test
    good_matches = []
    for m_n in matches:
        m, n = m_n
        if m.distance < ratio * n.distance:
            good_matches.append(m)

    return good_matches
```

```
In [5]: def homography(kp1, kp2, matches):
    # Estimate Homography with RANSAC
    src_pts = np.float32([kp1[m.queryIdx].pt for m in matches]).reshape(-1,1,2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in matches]).reshape(-1,1,2)
    H, mask = cv.findHomography(src_pts, dst_pts, cv.RANSAC, 5.0) # Compute Homography
    return H, mask
```

```
In [6]: def image_stich(img1, img2, H):
    # compute where the corners of img1 map to in img2 coordinates
    h1, w1 = img1.shape[:2]
    h2, w2 = img2.shape[:2]

    corners_img1 = np.float32([[0,0], [0,h1], [w1,h1], [w1,0]]).reshape(-1,1,2)
    warped_corners = cv.perspectiveTransform(corners_img1, H)

    # Combine both sets of corners to find the full panorama extents
    all_corners = np.concatenate((warped_corners, np.float32([[0,0],[0,h2],[w2,h2],[w2,0]]).reshape(-1,1,2)), a
    [xmin, ymin] = np.int32(all_corners.min(axis=0).ravel() - 0.5)
    [xmax, ymax] = np.int32(all_corners.max(axis=0).ravel() + 0.5)
```

```

# Compute translation to avoid negative coordinates
translation = [-xmin, -ymin]
T = np.array([[1, 0, translation[0]], [0, 1, translation[1]], [0, 0, 1]])

# Warp img1 into the panorama canvas
result = cv.warpPerspective(img1, T @ H, (xmax - xmin, ymax - ymin))

# Paste img2 into the panorama
result[translation[1]:h2 + translation[1], translation[0]:w2 + translation[0]] = img2

return result

```

```

In [7]: # extract SIFT features
sift = cv.SIFT_create()

# SIFT descriptors
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
kp3, des3 = sift.detectAndCompute(img3, None)

# matches
match12 = find_match(des1, des2)
match23 = find_match(des3, des2)

# homography
H12, mask12 = homography(kp1, kp2, match12)
H23, mask23 = homography(kp3, kp2, match23)

# image stitching
result12 = image_stich(img1, img2, H12)
result23 = image_stich(img3, img2, H23)

```

Print Homography Matrix

```

In [9]: print(H12)
print(H23)

[[ 2.18131489e+00  3.62680304e-01 -5.14170932e+03]
 [ 5.12066031e-01  2.06083271e+00 -1.35267252e+03]
 [ 3.86268917e-04  3.29907388e-05  1.00000000e+00]]
[[ 6.03155195e-01 -4.14187746e-02  1.48992000e+03]
 [-1.69227067e-01  8.61497689e-01  1.68608408e+02]
 [-1.25305924e-04 -9.70391702e-06  1.00000000e+00]]

```

Output Image

```

In [10]: kp4, des4 = sift.detectAndCompute(result12, None)
kp5, des5 = sift.detectAndCompute(result23, None)
match45 = find_match(des4, des5)
h45, mask45 = homography(kp4, kp5, match45)

result45 = image_stich(result12, result23, h45)

```

```

In [11]: plt.imshow(result45)
plt.title("final panaroma of my living room")
plt.show()

```



Exercise 2

```
In [12]: img4 = cv.cvtColor(cv.imread('./hw10_image/ex2_0.jpg'), cv.COLOR_BGR2RGB)
plt.imshow(img4)
plt.show()
```

